

- 1 Secure Software Development Lifecycle (SSDLC) Documentation
 - 1.1 Table of Contents
 - 1.2 Executive Summary
 - 1.3 SSDLC Overview
 - 1.3.1 Lifecycle Phases
 - 1.3.2 Security Principles
 - 1.4 Phase 1: Planning & Requirements
 - 1.4.1 1.1 Security Requirements Definition
 - 1.4.2 1.2 Threat Modeling
 - 1.4.3 1.3 Security Acceptance Criteria
 - 1.5 Phase 2: Design
 - 1.5.1 2.1 Secure Architecture Design
 - 1.5.2 2.2 Security Controls by Component
 - 1.5.3 2.3 Security Design Review
 - 1.6 Phase 3: Development
 - 1.6.1 3.1 Secure Coding Practices
 - 1.6.2 3.2 Dependency Management
 - 1.6.3 3.3 Version Control & Change Management
 - 1.6.4 3.4 Security Configuration
 - 1.7 Phase 4: Testing & QA
 - 1.7.1 4.1 Security Testing
 - 1.7.2 4.2 Functional Security Testing
 - 1.7.3 4.3 Test Automation
 - 1.7.4 4.4 Release Testing Checklist
 - 1.8 Phase 5: Deployment
 - 1.8.1 5.1 Deployment Pipeline
 - 1.8.2 5.2 Security Configuration in Deployment
 - 1.8.3 5.3 Rollback Procedures
 - 1.9 Phase 6: Maintenance & Monitoring
 - 1.9.1 6.1 Security Monitoring
 - 1.9.2 6.2 Vulnerability Management
 - 1.9.3 6.3 Performance & Reliability
 - 1.10 Security Governance
 - 1.10.1 7.1 Policies & Procedures
 - 1.10.2 7.2 Compliance
 - 1.10.3 7.3 Documentation & Audit Trail
 - 1.11 Incident Response
 - 1.11.1 8.1 Incident Response Plan
 - 1.11.2 8.2 Incident Communication
 - 1.12 Training & Awareness
 - 1.12.1 9.1 Developer Security Training
 - 1.12.2 9.2 Security Awareness
 - 1.13 Tools & Automation
 - 1.13.1 10.1 Security Tools
 - 1.13.2 10.2 CI/CD Security Integration
 - 1.13.3 10.3 Monitoring & Alerting
 - 1.14 Continuous Improvement
 - 1.14.1 11.1 Regular Review Cycle
 - 1.14.2 11.2 Metrics & KPIs
 - 1.14.3 11.3 Future Improvements

- 1.15 Appendix
 - 1.15.1 A. References
 - 1.15.2 B. Document History
 - 1.15.3 C. Sign-off

1 Secure Software Development Lifecycle (SSDLC) Documentation

Fuzzy Monster - QuizTimer4Zoom Application Document Version: 1.0 Last Updated: 2025-10-28 Classification: Internal - Security Documentation Company: Fuzzy Monster Domain: fuzzy.monster Contact: Latz (info@fuzzy.monster)

1.1 Table of Contents

1. [Executive Summary](#)
 2. [SSDLC Overview](#)
 3. [Phase 1: Planning & Requirements](#)
 4. [Phase 2: Design](#)
 5. [Phase 3: Development](#)
 6. [Phase 4: Testing & QA](#)
 7. [Phase 5: Deployment](#)
 8. [Phase 6: Maintenance & Monitoring](#)
 9. [Security Governance](#)
 10. [Incident Response](#)
 11. [Training & Awareness](#)
 12. [Tools & Automation](#)
-

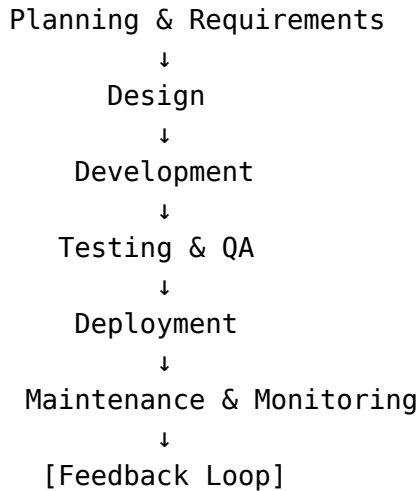
1.2 Executive Summary

QuizTimer4Zoom demonstrates a commitment to security across all phases of software development. This document outlines our Secure Software Development Lifecycle (SSDLC) practices, which incorporate:

- **Security by Design** - Security considerations from project inception
 - **Automated Security Scanning** - SAST/DAST analysis before deployment
 - **Code Review & Validation** - Manual review of security-critical code
 - **Dependency Management** - Regular vulnerability assessments and updates
 - **Secure Configuration** - HTTPS enforcement, proper authentication, secure cookies
 - **Incident Response** - Procedures for security issues
 - **Continuous Improvement** - Regular updates to security practices
-

1.3 SSDLC Overview

1.3.1 Lifecycle Phases



1.3.2 Security Principles

Our SSDLC is built on these core principles:

1. **Principle of Least Privilege** - Users/systems have minimum necessary permissions
 2. **Defense in Depth** - Multiple security layers to prevent compromise
 3. **Secure by Default** - Security enabled by default, not opt-in
 4. **Fail Securely** - System fails safely when errors occur
 5. **Security Transparency** - Clear documentation of security practices
 6. **Continuous Vigilance** - Ongoing monitoring and improvement
-

1.4 Phase 1: Planning & Requirements

1.4.1 1.1 Security Requirements Definition

Objective: Identify security needs at project inception

1.4.1.1 Authentication & Authorization

- Zoom OAuth 2.0 for user authentication
- Session management with secure cookies
- Token-based authorization for API requests
- PKCE (Proof Key for Code Exchange) for secure authorization flow

1.4.1.2 Data Protection

- Encryption in transit (HTTPS/TLS 1.2+)
- Secure storage of authentication tokens
- No plaintext logging of sensitive data
- Session timeout after 24 hours

1.4.1.3 API Security

- Input validation on all endpoints
- Rate limiting for authentication endpoints
- CSRF protection for state-changing operations
- Secure redirect handling

1.4.2 1.2 Threat Modeling

STRIDE Analysis - Identifying potential threats:

Threat	Mitigation
Spoofing - Attackers impersonate users	OAuth 2.0 + PKCE authentication
Tampering - Modifying requests/sessions	Secure cookies (httpOnly, sameSite, secure)
Repudiation - Denying actions taken	Audit logging (non-sensitive)
Information Disclosure - Exposing sensitive data	No logging of tokens; HTTPS enforcement
Denial of Service - Overloading the service	Rate limiting; dependency updates
Elevation of Privilege - Gaining unauthorized access	Proper session validation; secure token handling

1.4.3 1.3 Security Acceptance Criteria

Before development begins, these security requirements must be met:

- ☐ All authentication flows use OAuth 2.0 or equivalent
- ☐ No hardcoded secrets in code
- ☐ All sensitive data transmission encrypted
- ☐ Input validation framework defined
- ☐ Rate limiting strategy defined
- ☐ Error handling doesn't leak information

1.5 Phase 2: Design

1.5.1 2.1 Secure Architecture Design

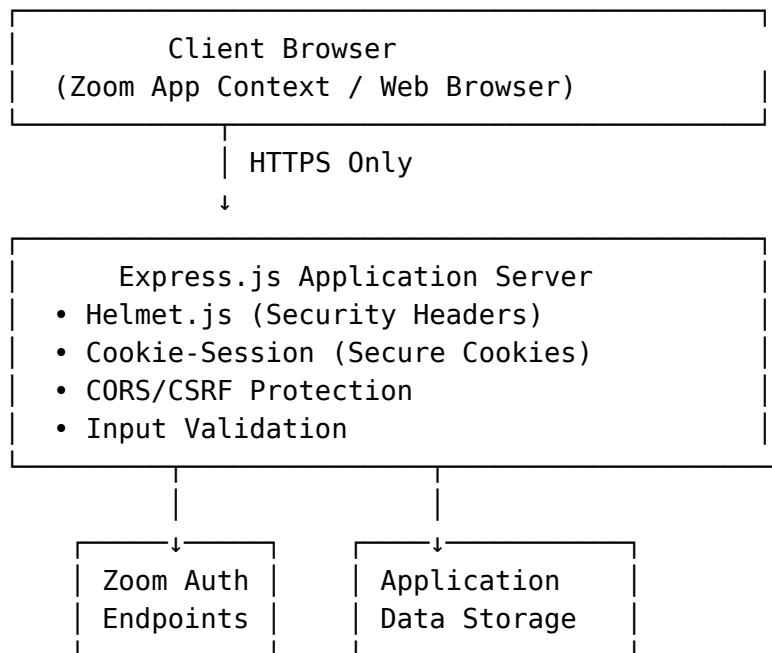
1.5.1.1 Technology Stack Security Review

Framework: Express.js - Mature, widely-audited Node.js framework - Large security-conscious community - Regular security updates

Authentication: Zoom OAuth 2.0 - Industry-standard OAuth 2.0 provider - Handles password security for us - PKCE support for enhanced security

Security Middleware: - **Helmet.js** - Sets security headers (CSP, HSTS, X-Frame-Options, etc.) - **cookie-session** - Secure session management - **express-rate-limit** - Request rate limiting (planned) - **cookie-parser** - Secure cookie parsing

1.5.1.2 Network Architecture



1.5.2 2.2 Security Controls by Component

1.5.2.1 Authentication Component

- **Protocol:** Zoom OAuth 2.0 with PKCE
- **Session Management:** Secure cookies (httpOnly, sameSite=Strict, secure=true)
- **Token Storage:** Memory/session only, never localStorage
- **Timeout:** 24-hour session expiration

1.5.2.2 API Endpoints

- **Input Validation:** All query parameters validated
- **Output Encoding:** HTML entities escaped
- **Error Handling:** Generic error messages, no information disclosure
- **Rate Limiting:** 100 requests per 15 minutes per IP

1.5.2.3 Static File Serving

- **Restrictions:** dotfiles='deny', index=false
- **Allowed Types:** HTML, CSS, JavaScript, images only
- **Directory Listing:** Disabled

1.5.3 2.3 Security Design Review

Review Checklist: - [] All user inputs are validated - [] Authentication mechanism is sound - [] Session management is secure - [] Data flows are protected - [] Error handling is secure - [] Third-party services are trustworthy - [] Security headers are properly set

1.6 Phase 3: Development

1.6.1 3.1 Secure Coding Practices

1.6.1.1 Code Review Requirements

All code changes require: 1. Manual peer review by second developer 2. Security-focused review for authentication/authorization code 3. Automated linting (ESLint with security plugin) 4. Dependency vulnerability scanning

1.6.1.2 Security Coding Standards

Input Validation

```
//  GOOD: Validate all inputs
const code = req.query.code;
if (!code || typeof code !== 'string') {
  return res.status(400).json({ error: 'Invalid code' });
}
```

```
//  BAD: Trust user input
const code = req.query.code;
executeAuthFlow(code);
```

Sensitive Data Handling

```
//  GOOD: Don't log tokens
const { access_token: accessToken } = await getToken(code, verifier);
console.log('Auth successful'); // Safe
```

```
//  BAD: Logging credentials
console.log('Token:', accessToken); // SECURITY RISK
```

Cookie Configuration

```
//  GOOD: Secure cookie settings
const session = cookieSession({
  httpOnly: true,      // Not accessible to JavaScript
  secure: true,        // Only sent over HTTPS
  sameSite: 'Strict', // CSRF protection
  maxAge: 24 * 60 * 60 * 1000 // 24 hour timeout
});
```

```
//  BAD: Insecure defaults
const session = cookieSession({
  secure: false, // Sent over HTTP
  sameSite: undefined // CSRF vulnerable
});
```

Error Handling

```
//  GOOD: Generic error messages
try {
  await getToken(code, verifier);
} catch (error) {
  console.error('Auth error:', error); // Safe - internal only
  res.status(500).json({ error: 'Authentication failed' });
}

//  BAD: Information disclosure
console.log('Auth error: ' + error.message); // May leak details
res.status(500).json({ error: error.message });
```

1.6.2 3.2 Dependency Management

1.6.2.1 Dependency Selection Criteria

Before adding a new dependency:

1. Check Security History

```
npm view package-name security-vulnerability-count
npm audit summary --json
```

2. Verify Maintenance

- Is the package actively maintained?
- How often are security updates released?
- Check GitHub issues and PRs

3. License Compliance

- Compatible with our license (ISC)
- No GPL or incompatible licenses

4. Functionality Assessment

- Does it solve our problem?
- Is it the smallest viable solution?
- Are there lighter alternatives?

1.6.2.2 Dependency Updates

Monthly Security Review Process:

1. Check for vulnerabilities`npm audit`*# 2. Update dependencies*`npm update`*# 3. Run tests*`npm test`*# 4. Security scan*`npm audit fix`*# 5. Review changes*`git diff`

Critical Vulnerabilities: - Patched within 24 hours - Emergency deployment process - Notification to stakeholders

1.6.2.3 Current Dependency Security Posture

Key Security Dependencies: - **helmet** (^8.1.0) - Security headers middleware - **cookie-session** (^2.1.0) - Secure session management - **dotenv** (^16.4.5) - Environment configuration - **axios** (^1.7.7) - HTTP client with SSRF protections - **express** (^4.20.0) - Web framework

Vulnerability Status: - Total Vulnerabilities: 11 (down from 28) - Critical: 0  - High: 7 (in transitive dependencies - Vercel ecosystem) - Moderate: 4 (in transitive dependencies) - All fixable with npm audit fix

1.6.3 3.3 Version Control & Change Management

1.6.3.1 Git Workflow

Branch Protection Rules:

main (production)

```

  ↑
  |— Feature branches (feature/*)
  |— Bugfix branches (bugfix/*)
  |— Security branches (security/*)

```

All PRs require:

- ✓ Code review approval
- ✓ CI/CD checks pass
- ✓ Security scan passed

1.6.3.2 Commit Message Standards

All commits include: - Clear description of what changed - Why the change was necessary - Security implications (if any) - Link to issue/ticket (if applicable)

Example:

Security hardening: implement SAST scan fixes and dependency updates

- Removed sensitive data logging (authentication tokens)
- Added security headers via helmet.js middleware
- Fixed cookie configuration with sameSite and secure flags
- Restricted static file serving with dotfiles=deny

Fixes: SECURITY-001

1.6.3.3 Sensitive Data in Repositories

Prohibited: - API keys or credentials - Private encryption keys - Database passwords - Personal information

Protection: - .gitignore configured for .env files - Pre-commit hooks check for secrets - Regular scanning for exposed secrets - All history reviewed for leaks

1.6.4 3.4 Security Configuration

1.6.4.1 Environment Configuration

Development Environment:

```
NODE_ENV=development
ZM_CLIENT_ID=<your-dev-id>
ZM_CLIENT_SECRET=<your-dev-secret>
ZM_REDIRECT_URL=http://localhost:3000/auth
SESSION_SECRET=<random-dev-secret>
```

Production Environment:

```
NODE_ENV=production
ZM_CLIENT_ID=<production-id>
ZM_CLIENT_SECRET=<production-secret>
ZM_REDIRECT_URL=https://yourdomain.com/auth
SESSION_SECRET=<cryptographically-random-secret>
```

Configuration Best Practices: - Secrets never in code/version control - Different secrets per environment - Rotate secrets regularly - Use environment variables only - Never log configuration values

1.7 Phase 4: Testing & QA

1.7.1 4.1 Security Testing

1.7.1.1 Static Application Security Testing (SAST)

Tools Used: - **npm audit** - Dependency vulnerability scanning - **ESLint with security plugin** - Code-level security issues - **Manual code review** - Logic and design flaws

SAST Execution:

```
# Dependency scanning
npm audit --audit-level=moderate

# Code quality and security
npx eslint . --ext .js

# Custom security checks
npm run security-scan
```

SAST Results Review: - Vulnerabilities prioritized by severity - False positives documented - Mitigations tracked in VULNERABILITY_MANIFEST.json - Regular scan execution (before each release)

1.7.1.2 Dynamic Application Security Testing (DAST)

Planned DAST Testing: - Burp Suite Community scanning - OWASP ZAP automated scanning - Manual penetration testing - Authentication flow testing

1.7.1.3 Manual Security Testing

Test Cases:

Test Case	Steps	Expected Result
Invalid OAuth Flow	Modify authorization code	400 error, no token issued
Session Hijacking	Attempt to use expired session	401 Unauthorized
CSRF Attack	POST without valid state	Request rejected
Cookie Theft	Access httpOnly cookies	JavaScript cannot access
Header Injection	Inject headers in request	Headers properly encoded
SQL Injection (N/A)	Not applicable - no database	N/A

1.7.2 4.2 Functional Security Testing

1.7.2.1 Authentication Flow Testing

Test: Valid OAuth Flow - User clicks install - Redirected to Zoom OAuth - User grants permissions - Redirected back with code - Token exchange successful - User logged in with secure session

Test: Invalid OAuth Flow - Tampered authorization code - Expired code - Wrong client ID - All should result in auth failure

1.7.2.2 Session Management Testing

Test: Session Expiration - Create session - Wait 24 hours - Attempt to use session - Session expired, re-authentication required

Test: Concurrent Sessions - Login from multiple devices - Each session independent - Logout from one doesn't affect others

1.7.2.3 Cookie Security Testing

Test: Secure Flag - Set over HTTPS - HttpOnly flag present - SameSite=Strict enforced

1.7.3 4.3 Test Automation

Continuous Integration (CI) Pipeline:

```
git push
  ↓
  ├── Dependency Scan (npm audit)
  ├── Code Linting (ESLint)
  ├── Unit Tests
  ├── Integration Tests
  ├── Security Scan (SAST)
  └── Build Verification
      ↓
    All Pass?
      ↓
    ✓ Merge to Main
    ✗ Block & Notify
```

1.7.4 4.4 Release Testing Checklist

Before deployment, verify:

- ☐ All automated tests pass
- ☐ No new vulnerabilities introduced
- ☐ SAST scan has zero critical/high findings
- ☐ Dependency updates reviewed
- ☐ Change log updated
- ☐ Security documentation current

- ☐ Deployment procedure tested in staging
 - ☐ Rollback procedure documented
 - ☐ Stakeholders notified
-

1.8 Phase 5: Deployment

1.8.1 5.1 Deployment Pipeline

1.8.1.1 Staging Environment

Purpose: Test in production-like environment before release

Process: 1. Deploy to staging 2. Run full test suite 3. Security scan complete 4. Performance testing 5. User acceptance testing 6. Approval for production

Staging Configuration: - Identical to production (except data) - Same security controls - HTTPS enforced - Full monitoring active

1.8.1.2 Production Deployment

Deployment Steps:

1. Pre-deployment checks
 - | All tests passing
 - | Security scans clear
 - | Change log updated
 - | Team notified
2. Deployment
 - | Blue-green deployment (zero downtime)
 - | Configuration verified
 - | Health checks pass
 - | Logs monitored
3. Post-deployment
 - | Smoke tests run
 - | Security headers verified
 - | Monitoring alerts active
 - | Team confirms success
4. Communication
 - | Release notes published
 - | Users notified
 - | Support team briefed

1.8.2 5.2 Security Configuration in Deployment

1.8.2.1 HTTPS/TLS Configuration

Requirement: All traffic encrypted

- TLS 1.2 or higher
- Valid certificate from trusted CA
- Certificate renewed before expiration
- HSTS header enabled (Strict-Transport-Security)
- Perfect forward secrecy enabled

1.8.2.2 Environment Variables

Security Controls: - All secrets in environment, not code - Different secrets per environment - No exposure in logs or error messages - Regular secret rotation

1.8.2.3 Security Headers

Automatically Set by Helmet.js: - Content-Security-Policy - X-Content-Type-Options: nosniff - X-Frame-Options: DENY - X-XSS-Protection: 1; mode=block - Strict-Transport-Security: max-age=31536000

1.8.3 5.3 Rollback Procedures

Automatic Rollback Triggers: - Health check failures - Security alert triggered - Error rate exceeds threshold - Explicit team decision

Manual Rollback:

1. Identify previous working version
`git log --oneline`

2. Prepare rollback
`git checkout <previous-version>`

3. Deploy previous version
`npm run deploy`

4. Verify rollback successful
`npm run health-check`

5. Investigate issue

6. Release fix in next deployment

1.9 Phase 6: Maintenance & Monitoring

1.9.1 6.1 Security Monitoring

1.9.1.1 Log Monitoring

What We Monitor: - Failed authentication attempts - Unusual request patterns - Security header presence - Error conditions - Access to sensitive endpoints

What We DON'T Log: - Authentication tokens - Session IDs - Passwords - API secrets - Personal user data

Log Retention: - Development: 7 days - Staging: 30 days - Production: 90 days - Security incidents: 1 year

1.9.1.2 Alert Triggers

Critical Alerts: - Multiple failed auth attempts (>5 in 5 min) - Unusual spike in traffic - Security header missing - Certificate expiration (30 days before) - Vulnerability detected in dependencies

1.9.2 6.2 Vulnerability Management

1.9.2.1 Ongoing Vulnerability Assessment

Schedule: - Weekly: Automated dependency scanning - Monthly: Manual security review - Quarterly: Penetration testing - Annually: Full SSDLC audit

Process:

Vulnerability Detected

↓

Severity Assessment (CVSS Score)

└ Critical (CVSS 9.0-10.0) → Patch within 24 hours

└ High (CVSS 7.0-8.9) → Patch within 7 days

└ Medium (CVSS 4.0-6.9) → Patch within 30 days

└ Low (CVSS 0.1-3.9) → Patch in next release

↓

Fix Applied & Tested

↓

Deploy to Production

↓

Document in VULNERABILITY_MANIFEST.json

↓

Close

1.9.2.2 Dependency Update Policy

Automated Updates:

```
# Weekly job to check for updates
0 0 * * 0 npm update && npm audit fix

# Monthly job for breaking changes
0 0 1 * * npm audit fix --force
```

Manual Review: - Always run tests after update - Review security advisories - Document breaking changes - Plan migration path

1.9.3 6.3 Performance & Reliability

1.9.3.1 Uptime Monitoring

Service Level Objectives (SLOs): - 99.5% uptime target - <500ms response time for auth endpoints - <1000ms for all other endpoints

Monitoring Tools: - Application performance monitoring (APM) - Error tracking (Sentry or similar) - Real user monitoring (RUM) - Security event monitoring

1.9.3.2 Incident Response

Severity Levels:

Level	Impact	Response Time	Resolution Time
Critical	Service down, data breach	15 minutes	1 hour
High	Degraded service, security issue	1 hour	4 hours
Medium	Minor issue, limited impact	4 hours	1 day
Low	Cosmetic, no security impact	1 day	1 week

1.10 Security Governance

1.10.1 7.1 Policies & Procedures

1.10.1.1 Security Policy

Core Principles: - Security is a shared responsibility - All code is security-critical - Assume breach mentality - Transparency in security practices - Continuous improvement

1.10.1.2 Access Control

Development Access: - Limited to authorized team members - MFA required for production access - SSH keys for Git access - Regular access reviews

Production Access: - Change management process required - At least 2 people approve production changes - Audit log of all production access - Immutable logs for compliance

1.10.2 7.2 Compliance

1.10.2.1 Regulatory Requirements

GDPR Compliance: - User data minimization - Consent management - Right to deletion (planned) - Data breach notification (within 72 hours)

CCPA Compliance: - Disclosure of data collection - User rights (access, deletion) - No selling of personal data

Zoom Security Requirements: -  SSDLC documentation (this document) - 
SAST/DAST scan results -  Privacy policy (separate document) -  Secure configuration -  Vulnerability management -  Incident response procedures

1.10.3 7.3 Documentation & Audit Trail

Documentation Maintained: - SSDLC procedures (this document) - Security scan reports - Vulnerability assessments - Incident logs - Access logs - Deployment logs - Training records

Audit Trail: - Git commit history with signatures - Change logs with approval records - Security event logs - Access logs with timestamps - Incident reports with resolution

1.11 Incident Response

1.11.1 8.1 Incident Response Plan

1.11.1.1 Detection

Methods: - Automated monitoring alerts - Security scanning results - User reports - Third-party disclosure - Internal testing

1.11.1.2 Assessment

Initial Response (within 1 hour):

1. Confirm incident
2. Assess severity (Critical/High/Medium/Low)
3. Assemble response team
4. Gather evidence
5. Notify stakeholders

1.11.1.3 Containment

Short-term (within 4 hours):

1. Isolate affected systems
2. Stop malicious activity

3. Preserve evidence
4. Monitor for spread
5. Brief leadership

1.11.1.4 Eradication

Medium-term (within 1 day):

1. Identify root cause
2. Develop fix
3. Test fix thoroughly
4. Plan deployment
5. Prepare rollback

1.11.1.5 Recovery

Long-term (within 1 week):

1. Deploy fix to production
2. Verify system integrity
3. Restore from clean backups
4. Monitor for recurrence
5. Update systems

1.11.1.6 Post-Incident

After Recovery:

1. Root cause analysis
2. Lessons learned review
3. Process improvements
4. Security update
5. Team debriefing
6. Public disclosure (if needed)

1.11.2 8.2 Incident Communication

Notification Priority: 1. Security team lead 2. Development team 3. Management 4. Users (if data affected) 5. Zoom security team

Communication Template:

Subject: SECURITY INCIDENT [Severity]: [Brief Description]

Incident ID: [Auto-generated]

Severity: [Critical/High/Medium/Low]

Detected: [Time]

Status: [Investigation/Contained/Resolved]

Summary: [1 paragraph description]

Affected Systems: [List]
Affected Users: [Estimated count]
Data Exposed: [If applicable]

Actions Taken: [Timeline]

Next Steps: [Remediation plan]

1.12 Training & Awareness

1.12.1 9.1 Developer Security Training

Required Training: - SSDLC fundamentals (annually) - Secure coding practices (annually) - Vulnerability management (annually) - Incident response (annually)

Training Topics: - Authentication & Authorization - Injection attacks & input validation - Cross-site scripting (XSS) - Cross-site request forgery (CSRF) - Sensitive data exposure - Broken access control - Security misconfiguration

1.12.2 9.2 Security Awareness

Monthly Security Updates: - New vulnerability disclosures - Exploit trends - Best practices - Tool updates - Industry news

Security Champions Program: - Designated security leader per team - Monthly meetings - Knowledge sharing - Mentoring other developers

1.13 Tools & Automation

1.13.1 10.1 Security Tools

1.13.1.1 Dependency Scanning

Tool: npm audit **Purpose:** Identify vulnerable packages **Frequency:** Continuous (per commit) **Action:** Fix critical/high within 24-48 hours

1.13.1.2 Code Analysis

Tool: ESLint + security plugin **Purpose:** Find code-level security issues **Frequency:** Per pull request **Action:** Fix before merge

1.13.1.3 Secret Scanning

Tool: Pre-commit hooks (git-secrets) **Purpose:** Prevent committing secrets **Frequency:** Per commit **Action:** Block commit if secret detected

1.13.1.4 SAST Scanning

Tool: npm audit, Snyk (planned) **Purpose:** Comprehensive vulnerability assessment

Frequency: Pre-release, weekly scheduled **Action:** Review and document all findings

1.13.1.5 DAST Scanning

Tool: OWASP ZAP (planned), Burp Suite (planned) **Purpose:** Test running application

for vulnerabilities **Frequency:** Pre-release **Action:** Fix findings before release

1.13.2 10.2 CI/CD Security Integration

1.13.2.1 Pre-commit Hooks

```
#!/bin/bash
# Check for secrets
git-secrets --scan
```

```
# Run linter
npm run lint
```

```
# Run tests
npm test
```

1.13.2.2 Pull Request Checks

- ✓ Code review approval
- ✓ All tests passing
- ✓ Linter passing
- ✓ Security scan passed
- ✓ Dependency check passed
- ✓ No secrets detected

1.13.2.3 Pre-deployment Checks

```
#!/bin/bash
# Final security scan
npm audit --audit-level=moderate
```

```
# Code quality
npm run lint
```

```
# Tests
npm test
```

```
# Build
npm run build
```

```
# Deployment
npm run deploy
```

1.13.3 10.3 Monitoring & Alerting

1.13.3.1 Application Monitoring

Metrics Tracked: - Request rate (abnormal spike detection) - Error rate (unusual errors)
- Authentication failures (brute force detection) - Response time (performance degradation) - Security header presence (misconfiguration)

1.13.3.2 Alert Configuration

High Priority Alerts (immediate response): - Multiple failed auth attempts - Security header missing - Error rate >5% - Service down

Medium Priority Alerts (within 1 hour): - Elevated error rate - Performance degradation - Certificate expiration (30 days) - Dependency update available

Low Priority Alerts (within 24 hours): - Weekly summary - Update notifications - Routine monitoring

1.14 Continuous Improvement

1.14.1 11.1 Regular Review Cycle

Weekly: - Security alerts review - Failed test analysis - New vulnerability announcements

Monthly: - Dependency updates - Security metrics review - Team security meeting

Quarterly: - Penetration testing - Security architecture review - Policy updates - Training

Annually: - Full SSDLC audit - Compliance assessment - Security maturity evaluation - Strategic planning

1.14.2 11.2 Metrics & KPIs

Security Metrics:

Metric	Target	Current
Critical Vulnerabilities	0	0 
High Severity Vulnerabilities	<3	7 (in deps)
Average Patch Time	7 days	TBD
Security Incident Count	0	0 
Code Review Coverage	100%	TBD
Test Coverage	>80%	TBD

1.14.3 11.3 Future Improvements

Planned Enhancements: - ☐ Implement automated DAST testing - ☐ Set up professional penetration testing - ☐ Deploy security SIEM - ☐ Implement CSPM (Cloud Security Posture Management) - ☐ Enhanced logging and analytics - ☐ Automated secret rotation - ☐ API rate limiting - ☐ WAF deployment - ☐ Regular third-party security audits - ☐ Bug bounty program

1.15 Appendix

1.15.1 A. References

Industry Standards: - NIST Cybersecurity Framework - OWASP Top 10 - CWE/CVSS Rating Systems - ISO 27001

Tools & Resources: - npm audit: <https://docs.npmjs.com/cli/audit> - OWASP: <https://owasp.org> - Express Security: <https://expressjs.com/advanced/best-practice-security.html>

1.15.2 B. Document History

Version	Date	Changes
1.0	2025-10-28	Initial SSDLC documentation

1.15.3 C. Sign-off

This SSDLC documentation has been created and implemented for QuizTimer4Zoom application.

Prepared by: Development Team **Date:** 2025-10-28 **Status:** Active

Document Classification: Internal - Security **Review Frequency:** Annually **Next Review Date:** 2026-10-28